Computer Networks 288 (2026) 112645

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

AdaDoHTool: A test-time adaptive framework for identifying DoH tunneling tools with unknown resolvers☆

Yandu Yi [a], Xiaoyan Hu [a,b,c,*], Yang Miao [a], Guang Cheng [a,d], Ruidong Li [e], Hua Wu [a]

[a] School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China
[b] Purple Mountain Laboratories for Network and Communication Security, Nanjing 211111, China
[c] Engineering Research Center of Blockchain Application, Supervision and Management, Ministry of Education, Nanjing 211189, China
[d] Jiangsu Province Engineering Research Center of Security for Ubiquitous Network, Nanjing 211189, China
[e] Institute of Science and Engineering, Kanazawa University, Kakuma, Kanazawa 920-1192, Japan

ARTICLE INFO

Keywords:
DNS over HTTPS (DoH)
DoH tunneling identification
Unknown resolver
Test-time adaptation

ABSTRACT

With the development of Domain Name System (DNS) encryption technologies, DNS over HTTPS (DoH) has emerged as a mainstream encrypted protocol. It enhances the security and privacy of traditional DNS while also providing malicious actors with a new covert channel for tunneling. Existing DoH tunneling traffic detection methods suffer significant performance degradation when encountering unknown recursive resolvers due to the diversity of resolver padding strategies and configuration differences, resulting in insufficient model generalization and adaptability. We propose AdaDoHTool, a test-time adaptive identification method for DoH tunneling tools targeting unknown recursive resolvers to address this issue. AdaDoHTool uses a word embedding matrix generated from unidirectional TLS application data record length sequences as input, applies a TextCNN model to extract deep feature representations, and incorporates a feature–label memory bank with an online update mechanism. Combined with a KNN classifier and a dedicated loss function, AdaDoHTool enhances feature transferability and target-domain adaptability. Experimental results show that under target-domain conditions with no labeled data (0%) and with a small amount of labeled data (10%), AdaDoHTool achieves average accuracies of 0.71 and 0.85, respectively, improving upon other comparison methods by 7.41% and 25.62%, respectively. Moreover, the method maintains low memory consumption. Its inference time is only 30.5% of the fastest baseline model.

1. Introduction

As a fundamental component of Internet infrastructure, the Domain Name System (DNS) provides users with convenient domain name resolution services, enabling the efficient operation of modern network applications. However, traditional DNS lacks encryption and authentication mechanisms during transmission, making it highly vulnerable to tunneling attacks. Therefore, the Internet Engineering Task Force (IETF) has proposed several encrypted DNS protocols, among which DNS over HTTPS (DoH) was standardized in RFC 8484 [1] in 2018. By encapsulating DNS queries and responses within HTTPS sessions, DoH effectively prevents third-party eavesdropping and tampering, significantly enhancing DNS security.

While DoH offers substantial privacy and security benefits, attackers exploit its encrypted nature to establish covert communication channels [2]. Particularly in command-and-control (C&C) and data exfiltration, DoH-based tunneling demonstrates high stealthiness. Malware such as Godlua [3] has been confirmed to use DoH for communicating with attacker-controlled servers. These threats have considerably increased the difficulty of network defense, highlighting the urgent need for accurate identification of tunneling tools within DoH traffic.

Existing studies [4,5] have achieved promising results in detecting DoH traffic and DoH tunneling traffic. Our previous work [6] achieves early detection of DoH traffic and early identification of DoH tunneling tool traffic under dynamic network conditions. However, most existing

☆ This research was supported in part by National Key R&D Program of China under Grant 2023YFB3106801, in part by National Natural Science Foundation of China Project under Grant 62472087, in part by Jiangsu Province Natural Science Foundation Project under Grant BK20231413, and in part by the Fundamental Research Funds for the Central Universities under Grant 2242025K30025.

* Corresponding author at: School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China.
E-mail addresses: yandu@seu.edu.cn (Y. Yi), xyhu@njnet.edu.cn (X. Hu), 220225084@seu.edu.cn (Y. Miao), gcheng@njnet.edu.cn (G. Cheng), liruidong@ieee.org (R. Li), hwu@njnet.edu.cn (H. Wu).

https://doi.org/10.1016/j.comnet.2026.112645
Received 3 December 2025; Received in revised form 24 May 2026; Accepted 2 August 2026
Available online 6 August 2026
1389-1286/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

methods do not consider the impact of the diversity of DoH recursive resolvers. Different resolvers may lead to significant variations in traffic patterns, resulting in distribution shifts between the source and target domains. In real-world deployment, models are often required to handle DoH resolvers that are unseen during training, which may cause substantial performance degradation. Such differences may arise from multiple factors, including padding strategies, resolver implementations, deployment locations, and server-side configurations. For example, the DoH protocol does not mandate a standardized padding strategy. Instead, it only recommends the use of the EDNS(0) padding extension defined in RFC 7830 [7] and RFC 8467 [8]. These factors further increase the diversity of DoH traffic patterns. In addition, most existing DoH tunneling detection and identification models follow an offline training paradigm. When traffic from new resolvers appears, these models usually require newly collected labeled data and retraining from scratch, which limits their real-time responsiveness.

Therefore, this paper focuses on two practical problems: improving the model's generalization ability to diverse traffic patterns introduced by different resolvers, and enabling the model to update itself using newly observed traffic without retraining from scratch.

Generalization. Generalization refers to the ability of a model to maintain stable identification performance when encountering unseen DoH resolvers during deployment.

Online Update. Online update refers to the ability of a model to dynamically adjust its decision capability using continuously arriving target-domain test samples during deployment, without retraining the entire model.

To address these challenges, we introduce Test-Time Adaptation (TTA) into DoH tunneling tool identification under unknown resolvers and propose a task-specific adaptive method named AdaDoHTool. Unlike traditional offline training methods, AdaDoHTool does not require access to target-domain data during training. Instead, it adapts online using target samples that arrive during inference. To improve cross-resolver generalization and online update capability, AdaDoHTool incorporates two task-oriented designs. First, it adopts KNN-aware representation learning to enhance feature transferability, allowing the model to preserve a more reliable neighborhood structure and class separability under resolver-induced distribution shifts. Second, it constructs a dynamically updated memory bank and combines it with KNN-based similarity-weighted voting, enabling the model to gradually absorb target-domain information and adjust the effective decision boundary without updating the backbone parameters.

Specifically, AdaDoHTool uses a word embedding matrix generated from unidirectional TLS application data record length sequences as input and adopts TextCNN as the backbone network. A KNN-aware loss function is designed to enhance the backbone, enabling it to extract transferable feature representations from the embedding matrix. Then, a feature–label memory bank with an online update mechanism is constructed. The memory bank is initialized with source-domain feature–label pairs during training and dynamically updated with high-confidence target samples during inference. A KNN-based classifier is used for label prediction, allowing the model to progressively adapt to the target-domain traffic distribution of unknown resolvers with limited labeled samples or even without labeled samples.

The main contributions are summarized as follows:

- We propose AdaDoHTool, an innovative test-time adaptation-based method for identifying DoH tunneling traffic. We construct a feature memory bank to store feature–label pairs. The memory bank is dynamically updated during the test phase. By integrating a KNN-based classifier with the memory bank, the model achieves online adaptation to the evolving traffic distribution in the target domain.
- We employ the TextCNN as the backbone network and incorporate a KNN-aware loss function to extract deep representations from the word embedding matrix. This design enables the model to learn more transferable representations and improves its generalization to the target domain.
- We conduct extensive comparative experiments to evaluate the effectiveness of the proposed method. Results demonstrate that AdaDoHTool achieves an accuracy of 85% in identifying tunneling traffic from unknown DoH resolvers with 10% labeled data and consistently outperforms other baselines.

The remainder of this paper is organized as follows. We introduce the related work in Section 2. Next, we present the research issues in Section 3. Then, we describe the details of AdaDoHTool in Section 4. Next, we evaluate the performance of AdaDoHTool in Section 5. We conduct a discussion in Section 6. Finally, we conclude this paper in Section 7.

2. Related work and background

This section first analyzes the existing work in DoH traffic detection and DoH tunneling detection, and then introduces the Test-Time Adaptation concept adopted by our AdaDoHTool. Table 1 summarizes recent work and our AdaDoHTool.

2.1. DoH traffic detection

DoH traffic detection aims to distinguish DoH traffic from general HTTPS traffic. Most existing methods rely on handcrafted features combined with machine learning classifiers. Vekshin et al. [4] proposed features such as flow duration and packet size variance, and selected 18 key features using Gini index-based feature importance ranking. They evaluated five classifiers, including KNN and Random Forest (RF), achieving over 99% accuracy in both DoH detection and DoH client classification tasks. Wu et al. [9] used TLS record length sequences and direction features with AutoEncoder models to detect DoH traffic, and further verified flows through IP and SNI analysis. Bannat Wala et al. [10] collected traffic from three public DoH resolvers across different geographic locations, removed IP-related features, retained 17 attributes, and applied PCA and t-SNE for visualization-based detection. Their PCA-based method achieved an average accuracy of up to 99%. Jerabek et al. [11] reproduced and evaluated seven published methods on standardized datasets to assess their transferability and practicality.

2.2. DoH tunneling traffic detection

Research on DoH tunneling traffic detection primarily relies on machine learning and deep learning models, combined with statistical feature analysis. Most existing studies are based on the CIRA-CIC-DoHBrw-2020 [12] dataset and adopt offline supervised learning to improve detection accuracy.

MontazeriShatoori et al. [13] proposed a two-stage detection framework that combines statistical and time-series features. Their approach integrates LSTM with six traditional machine learning models and introduces the widely adopted CIRA-CIC-DoHBrw-2020 dataset. Nguyen and Park [14] proposed a semi-supervised DoH tunnel detection system based on Transformer models, requiring only 20% of the labeled data used in other approaches. Using 34 statistical features extracted with CICFlowmeter [15], their system demonstrates strong detection performance with reduced annotation cost. Zebin et al. [5] introduced a balanced stacked Random Forest model. They analyzed feature distributions using Kernel Density Estimation (KDE) and employed the SHAP (Shapley Additive Explanations) library to visualize and interpret model decisions, enhancing model explainability. Zhan et al. [16] analyzed the TLS fingerprint characteristics of DoH clients, including TLS record lengths and inter-packet intervals, and constructed classifiers based on flow-level features. They also evaluated the impact of variables such as server geolocation, DoH resolver type, EDNS(0) padding, and query timing on detection performance.

Moure-Garrido et al. [17] designed a real-time detection system for malicious DoH tunnels. By analyzing behavioral differences between

Table 1
Comparison of existing methods and AdaDoHTool.

Granularity	Method	Detection technique	Traffic representation	Performance	
				Generalization	Online update
DoH Traffic Detection	Vekshin et al.	KNN/RF/AdaBoost	18 statistical features	✗	✗
	DOHUNTER	AutoEncoder	TLS record length	✔	✗
	Bannat et al.	PCA/t-SNE/UMAP	17 statistical features	✗	✗
DoH Tunneling Detection	Nguyen et al.	Transformer	34 statistical features extracted with CICFlowmeter	✗	✗
	Zebin et al.	Balanced Stacked RF	29 statistical features	✗	✗
	Zhan et al.	whitelist mechanism & LR/DT/RF	TLS fingerprinting & TLS record length and time intervals	✗	✗
	Moure-Garrido et al.	Threshold-Based IDS	37 statistical features	✗	✔
	METC	CNN-BiGRU-Attention	28 statistical features extracted using enhanced DoHlyzer	✔	✗
	WEDoHTool	TextCNN + Transformer	TLS application data record length	✗	✗
	AdaDoHTool(ours)	**Test-Time Adaptation**	**TLS application data record length**	✔	✔

benign and malicious flows, they defined threshold-based heuristics using packet length and inter-packet intervals to identify malicious connections rapidly. In follow-up work, Moure-Garrido et al. [18] proposed a lightweight, per-packet detection method that monitors abnormal traffic behaviors in real time. Using statistical analysis, they extracted key features such as concurrent connections and inter-connection intervals. The system achieves 97.30% accuracy while maintaining low computational overhead. Zuo et al. [19] extracted statistical features like flow duration and designed a hybrid model combining Convolutional Neural Networks (CNN), bidirectional GRU, and multi-head attention to capture local and temporal dependencies. They further leveraged transfer learning to enhance model robustness and generalization.

In our previous work [6], we proposed WEDoHTool, a word embedding-based method for early identification of DoH tunnel tooling traffic in dynamic network environments. We employed the Word2Vec technique to generate embeddings of TLS record length sequences and designed a two-stage identification module. However, WEDoHTool still follows an offline supervised learning paradigm and is primarily designed under the assumption that the target domain is consistent with the training domain. In contrast, AdaDoHTool inherits the idea of using TLS record length sequences and word embedding representations, but further extends the problem to the unknown-resolver scenario and improves cross-resolver generalization through test-time adaptation.

In conclusion, most existing DoH tunneling detection methods rely on statistical features, sequential features, and deep learning models, and usually adopt an offline training paradigm. Although some studies [16] have analyzed the influence of DoH resolver type, server location, and query timing, these factors are mostly treated as experimental variables rather than being explicitly modeled as distribution shifts. Unlike these methods, AdaDoHTool formulates DoH tunneling tool identification under unknown resolvers as a test-time adaptation problem. The key distinction is that AdaDoHTool dynamically updates a feature–label memory bank and adjusts the effective decision boundary to achieve online adaptation to target-domain traffic distributions. In addition, most existing DoH studies focus on tunneling detection, whereas this work further targets fine-grained tunneling tool identification.

2.3. Test-time adaptation

Domain Adaptation (DA) and Test-Time Adaptation (TTA) are introduced to overcome the domain shift problem. DA usually requires access to target-domain data during training in order to transfer knowledge from the source domain to the target domain. In contrast, TTA adapts the model only during inference by using incoming target samples.

Existing DA methods mainly include distribution-matching methods and adversarial-training methods. Distribution-matching methods learn domain-invariant representations by minimizing the discrepancy between source and target feature distributions. Representative methods include Deep Adaptation Network (DAN) [20] and Deep Subdomain Adaptation Network (DSAN) [21]. Adversarial-training methods introduce a domain discriminator to adversarially train the feature extractor, making source-domain and target-domain features difficult to distinguish. Representative methods include Domain-Adversarial Neural Network (DANN) [22] and Conditional Domain Adversarial Networks (CDANs) [23]. Recently, TTA has attracted increasing attention. Parameter-updating methods such as Tent [24] update normalization statistics or affine parameters during testing, while prototype- or memory-based methods such as T3A [25] improve target-domain prediction by adjusting class prototypes or maintaining feature caches of test samples.

Although TTA has been widely studied in computer vision, its application to encrypted traffic analysis remains limited. Most existing DoH tunneling detection methods follow an offline training paradigm and implicitly assume that training and testing traffic share similar distributions. However, in real-world deployments, different DoH resolvers may adopt different padding strategies, which can change key feature distributions. Since traffic from unknown resolvers usually appears only after deployment, collecting sufficient target-domain data in advance and retraining the model is often impractical. Therefore, TTA provides a more suitable adaptation paradigm for online DoH tunneling tool identification.

Compared with major TTA paradigms, AdaDoHTool is more suitable for DoH tunneling traffic detection. Unlike parameter-updating methods, it keeps the backbone fixed during testing, avoiding computational overhead and model drift. Unlike typical prototype-based methods, it maintains a fine-grained feature–label memory bank rather than only class-level prototypes, and performs prediction through KNN-based similarity-weighted voting. By updating the memory bank with high-confidence target samples, AdaDoHTool adjusts the effective decision boundary without retraining, making it suitable for online DoH traffic detection under diverse resolvers and changing distributions.

3. Problem statement

To address the generalization challenge caused by distribution shifts across resolvers, we formulate DoH tunneling traffic identification as a cross-domain classification problem. Let the input space be denoted as $\mathcal{X}$ and the label space as $\mathcal{Y} = \{B, M_1, M_2, \ldots, M_n\}$, where B represents benign DoH traffic and M_k denotes malicious DoH traffic generated by tunneling tool k ($k = 1, 2, \ldots, n$). The source domain is defined as $D_S = \{(x_i^S, y_i^S)\}$, where $x_i^S \in \mathcal{X}$ represents a traffic sample and $y_i^S \in \mathcal{Y}$ is its corresponding label. The target domain is defined as $D_T = \{x_j^T\}$, which contains samples collected from unseen DoH resolvers. D_S and D_T share the same label space $\mathcal{Y}$. Their joint distributions differ due to resolver diversity. The objective is to learn a classifier $F : \mathcal{X} \rightarrow \mathcal{Y}$

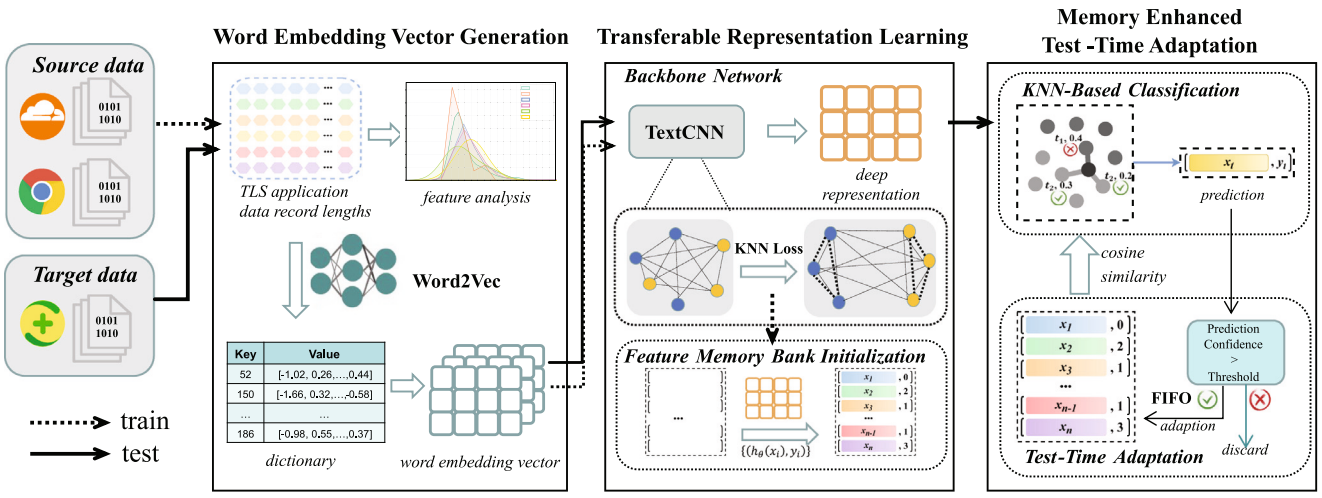


Fig. 1. Architecture overview of AdaDoHTool.

trained on $\mathcal{D}_S$ that can generalize to the unseen target domain $\mathcal{D}_T$. Formally, this can be expressed as:

$$F(x) = \arg\max_{y \in \mathcal{Y}} P(y \mid x; \theta), \quad x \in \mathcal{D}_T \tag{1}$$

where θ denotes the model parameters, and $P(y \mid x; \theta)$ is the probability of x belonging to class y under parameters θ. The challenge is to adapt a model trained on $\mathcal{D}_S$ to an unseen target distribution $\mathcal{D}_T$ for reliable DoH tunneling traffic identification under resolver shifts.

It should be noted that this formulation characterizes the cross-domain nature of the problem, which is consistent with the UDA (Unsupervised Domain Adaptation) setting. However, unlike UDA methods that require access to the target domain during training, AdaDoHTool follows a test-time adaptation paradigm. Samples from unknown resolvers are assumed to arrive during inference, and the model adapts to them online.

4. System design

This work proposes a test-time adaptive method for identifying DoH tunneling tool traffic under unknown resolvers. AdaDoHTool extracts word embedding vectors from TLS record length sequences, employs a KNN-aware TextCNN network to learn transferable features during source-domain training, and utilizes a KNN-based classifier with a memory bank to achieve test-time adaptation. This section systematically presents our proposed AdaDoHTool. First, we provide an overview of its architecture. Then, we elaborate on its embedding vector generation, transferable representation learning, and memory enhanced test-time adaptation.

4.1. Architecture overview

AdaDoHTool consists of three modules: word embedding vector generation, transferable representation learning, and memory enhanced test-time adaptation. The architecture of AdaDoHTool is illustrated in Fig. 1. Given a TLS application data record length sequence, AdaDoHTool first converts it into a word embedding matrix. Then, to enhance cross-resolver generalization, it learns transferable feature representations with a KNN-aware TextCNN backbone. Finally, to support online update during inference, it performs memory-enhanced KNN classification and dynamically updates the feature–label memory bank with high-confidence target-domain samples. The three modules are described as follows.

Word Embedding Vector Generation. This module converts TLS application data record length sequences into continuous vector representations using the Word2Vec model. By treating each length value as a word and the sequence as a sentence, the model captures contextual dependencies to enhance feature expressiveness.

Transferable Representation Learning. This module employs a TextCNN to extract deep representations from word embedding sequences without format conversion. To mitigate overfitting to the upstream task and enhance feature transferability, a KNN-aware backbone network is introduced. It preserves intra-class diversity and encourages inter-class separation by leveraging neighborhood similarity, aligning feature learning with the KNN-based inference strategy. This module also initializes a feature memory bank with feature–label pairs extracted from the source domain.

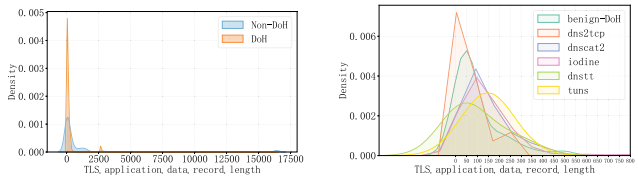
Memory Enhanced Test-Time Adaptation. This module employs a KNN-based classifier that retrieves the top-k nearest neighbors from the memory bank using cosine similarity, and predicts the final label through a similarity-weighted voting mechanism. During inference, the system continuously selects high-confidence test samples and dynamically updates the memory bank. As the testing process progresses, the feature distribution within the memory bank gradually shifts toward the target domain, enabling the model to achieve continuous adaptation to the target distribution without updating the backbone parameters.

4.2. Word embedding vector generation

In this module, we first extract the sequence of TLS application data record lengths of each unidirectional flow. Then, we employ word embedding techniques to transform the raw TLS length sequences into more informative representations.

4.2.1. TLS sequence features

The proposed method analyzes unidirectional flow features. This design is motivated by the widespread presence of asymmetric routing in real-world network environments, where certain backbone nodes may fail to capture complete bidirectional traffic flows. In addition, we focus on TLS-layer features rather than network-layer spatiotemporal features. Network-layer features, such as packet length sequences, are often unstable because they are affected by transmission uncertainty and variations in dynamic network conditions. Since TLS operates on top of TCP, it provides stronger guarantees in terms of transmission order and data independence, making TLS-derived features more reliable. Specifically, within the TLS protocol, our method uses the record-length sequence of application data as the traffic representation. Prior studies have shown that TLS record lengths have strong discriminative power for encrypted traffic analysis [26,27], which makes them suitable for identifying DoH tunneling tool traffic.



(a) DoH traffic vs. Non-DoH traffic **(b)** Benign DoH vs. Malicious DoH

Fig. 2. Kernel Density Estimation (KDE) of the first 15 TLS application data record lengths.

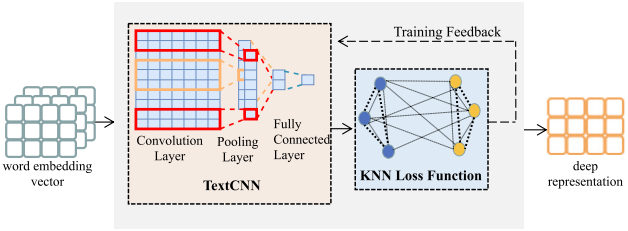


Fig. 3. KNN-aware TextCNN backbone network.

As illustrated in Fig. 2, Kernel Density Estimation (KDE) curves are used to depict the distributions of the first 15 TLS application data record lengths in two different stages. Fig. 2(a) compares DoH and non-DoH traffic, while Fig. 2(b) compares benign DoH traffic with various DoH tunneling tools. From Fig. 2(a), it can be observed that non-DoH traffic exhibits a broad distribution of TLS record lengths, primarily concentrated within the 0–1000 byte range, with a relatively uniform spread between 1000 and 2000 bytes. In contrast, DoH traffic demonstrates a more compact distribution, with most record lengths concentrated in the 0–500 byte interval. Fig. 2(b) further reveals the distinct differences in TLS record length distributions between benign DoH traffic and multiple types of tunneled DoH traffic. For example, benign DoH traffic shows a strong concentration of record lengths between 25 and 75 bytes. In contrast, dns2tcp exhibits two prominent density peaks near 0 and 260 bytes. Notably, the KDE curves of dnscat2 and iodine are highly similar, with record lengths primarily centered around 100 bytes, indicating a degree of feature similarity between these tools. Additionally, the TLS record lengths for dnstt and tuns follow approximately normal distributions, with peaks around 50 and 150 bytes.

In summary, although the first 15 TLS application data record lengths in unidirectional flows generally fall within specific ranges, distinct distribution patterns remain observable among different types of DoH traffic, making them suitable for classification tasks.

4.2.2. Word embedding representation

To mitigate performance degradation caused by sparse features or overfitting to local patterns, word embedding techniques are introduced to vectorize the TLS application data record lengths. These techniques not only preserve key semantic information in a low-dimensional space but also capture deep, latent structural dependencies within the sequence. This enhances the model's ability to represent complex traffic behaviors and improves its robustness. In this work, the TLS application data record length sequence is treated as a special form of "language". Each length value corresponds to a TLS application record, and the sequential nature of these records during encrypted communication exhibits contextual and patterned dependencies similar to the collocation of words in natural language.

The word embedding method adopted in this section is Word2Vec. Word2Vec provides two architectures: Continuous Bag-of-Words (CBOW) and Skip-gram. We employ the CBOW architecture for our implementation. First, a vocabulary is constructed from the observed TLS record lengths. Then, for each length sequence, its one-hot encoded form $\mathbf{w} = \{w_{t-m}, \ldots, w_{t-1}, w_t, w_{t+1}, \ldots, w_{t+m}\}$ is input into the CBOW model, which predicts the target word w_t based on its surrounding context words. The context window size is 2 m, meaning m words on each side of the target are used. The CBOW training objective is to maximize the log-probability of the target word given its context, defined by the following loss function:

$$\sum_{t=1}^{T} \log P(w_t \mid w_{t-m}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+m}) \tag{2}$$

Here, T is the number of tokens in the vocabulary. The conditional probability is defined as:

$$P(w_t \mid \text{context}) = \frac{\exp(u_{w_t} \cdot v_{\text{context}})}{\sum_{w \in V} \exp(u_w \cdot v_{\text{context}})} \tag{3}$$

where u_{w_t} is the output-layer weight vector for word w_t, u_w denotes the wth row of the hidden-to-output matrix, and v_{context} is the average embedding vector of the context words.

Word2Vec learns embeddings through unsupervised training. During this process, it maps each TLS record length to a vector representation. Each length functions as a word, and the entire sequence corresponds to a sentence. By maximizing the probability of predicting each target token from its context, the model produces embedding vectors that capture semantic relationships between record lengths.

4.3. Transferable representation learning

In this module, AdaDoHTool employs a TextCNN with a KNN-aware loss function to learn transferable representations for cross-resolver generalization. Since different DoH recursive resolvers may change TLS application data record length patterns, traffic from the same tunneling tool can exhibit resolver-specific feature variations. To address this issue, the KNN-aware loss optimizes the feature space from the perspective of nearest-neighbor retrieval, encouraging same-class samples to maintain reliable local relationships while keeping different classes separable. This helps target-domain samples from unseen resolvers retrieve semantically consistent neighbors from the memory bank during inference, thereby enhancing cross-resolver generalization. The learned features and their corresponding labels are then stored in the memory bank as reference samples for KNN-based classification.

4.3.1. Backbone network design

The overall framework of the proposed TextCNN backbone is illustrated in Fig. 3. Most existing TTA methods are primarily developed for computer vision tasks [28], where backbone architectures are commonly based on Residual Networks (ResNet). However, AdaDoHTool utilizes word embedding sequence matrices as input features. Applying ResNet directly in this context requires transforming these matrices into image-like formats to be compatible with convolutional structures, which introduces additional preprocessing overhead and may negatively affect model efficiency and deployment feasibility.

To address this issue, AdaDoHTool employs the TextCNN model as its backbone network. TextCNN is well-suited for processing word embedding matrices and has demonstrated effective feature extraction capabilities in related tasks. By adopting TextCNN, the proposed method avoids unnecessary format conversion, achieves better computational efficiency, and maintains consistency with the input data structure.

Moreover, when different recursive resolvers are used with the same DoH tunneling tool, variations in intra-class behavior may arise. Conventional supervised training tends to pull features with the same label closer in the feature space, a phenomenon known as overfitting to the upstream task. This may lead to the neglect of meaningful

intra-class semantic variations, thereby impairing model transferability [29].

To mitigate this limitation, AdaDoHTool introduces an optimized loss function based on the characteristics of the K-Nearest Neighbors (KNN) classifier. This function, denoted as L_{KNN}, is designed to replace the conventional cross-entropy loss function. Let the backbone network be represented as $h_\theta(\cdot)$, where θ is the set of model parameters. Define $B_{(\theta,D_S)}(x_i)$ as the ordered index set of samples from the source domain dataset D_S, ranked by their cosine similarity to the encoded representation $h_\theta(x_i)$ of input sample x_i. The source domain dataset is denoted as $D_S = \left\{(x_s^i, y_s^i)\right\}_{i=1}^{n_s}$, where each sample (x_s^i, y_s^i) is independently and identically sampled from the source distribution, and n_s is the total number of source samples. The KNN-aware loss function is defined as follows:

$$L_{\mathrm{KNN}} = \min_\theta \left(-\frac{1}{n_s} \sum_{i=1}^{n_s} \log \frac{N_i}{D_i} \right) \tag{4}$$

$$N_i = \sum_{j \in B_{(k,\theta,D_S)}(x_i)} \exp\left(\frac{w_{ij}}{\tau}\right) \cdot \mathbb{I}\{y_i = y_j\} \tag{5}$$

$$D_i = \sum_{j \in B_{(k,\theta,D_S)}(x_i)} \exp\left(\frac{w_{ij}}{\tau}\right) \tag{6}$$

Here, $B_{(k,\theta,D_S)}(x_i) \subseteq B_{(\theta,D_S)}(x_i)$ denotes the top-k subset of neighbors, and w_{ij} represents the cosine similarity between $h_\theta(x_i)$ and $h_\theta(x_j)$, computed as:

$$w_{ij} = \frac{h_\theta(x_i)^\top h_\theta(x_j)}{\|h_\theta(x_i)\|_2 \cdot \|h_\theta(x_j)\|_2} \tag{7}$$

The temperature hyperparameter $\tau > 0$ is used to scale the influence of similarity. The indicator function $\mathbb{I}\{y_i = y_j\}$ equals 1 if the labels match, and 0 otherwise.

This formulation enhances the similarity between features with the same label while preserving intra-class diversity and increasing inter-class separability. Compared to the conventional cross-entropy loss, the proposed L_{KNN} facilitates more transferable representations, as validated in prior research and supported by the ablation studies presented in this work.

4.3.2. Feature memory bank initialization

AdaDoHTool constructs a feature memory bank to store feature representations and their corresponding labels or predicted labels, obtained from both the source domain samples during training and the target domain samples during testing. These stored feature–label pairs serve as reference samples in subsequent inference steps, facilitating KNN-based classification.

At the initial stage of training, the memory bank is initialized as an empty set or array. During each training iteration, source domain samples are input to the backbone network to extract their feature representations. These are then paired with the corresponding ground-truth labels and added to the memory bank. After completing the full training process on the source domain, the memory bank contains a comprehensive set of feature–label pairs that represent the source domain, forming the foundation for inference in the target domain.

The size of the memory bank can be flexibly adjusted according to the total number of training samples. To balance training efficiency and model performance, the memory bank is denoted as $M = \left\{\left(h_\theta(x_i), y_i\right)\right\}_{i=1}^{n_s}$. During training, the memory bank is dynamically updated using a First-In First-Out (FIFO) policy. When new feature representations from a minibatch are added, and the memory bank reaches its capacity, the oldest entries are removed. This mechanism ensures that the memory bank consistently maintains the most up-to-date feature information, enabling the model to adapt to changes in the data distribution. As a result, the model's generalization ability and robustness in handling previously unseen domain data are significantly enhanced.

4.4. Memory enhanced test-time adaptation

This module employs a KNN-based classifier that predicts labels through similarity-weighted voting over feature–label pairs stored in the memory bank. To support online updates during inference, the memory bank is continuously updated with high-confidence target samples. Specifically, each incoming target sample is first predicted using the current memory bank. If its prediction confidence exceeds the predefined threshold, its feature representation and predicted label are added to the memory bank. The updated memory bank is then used for subsequent predictions, allowing AdaDoHTool to gradually incorporate target-domain information and adapt to the target distribution without modifying the backbone parameters.

4.4.1. KNN-based classification

The testing phase on the target domain begins once training on the source domain dataset has been completed. At this stage, a pretrained backbone network and a classifier are already available. For a given test sample x_u, the prediction is computed as follows:

$$\eta_k(x_u) = \mathrm{softmax}\left(\sum_{j \in B_{(k,\theta,M)}(x_u)} w_{uj} \cdot \mathbf{1}\{y_j\} \right) \tag{8}$$

In this equation, $\mathbf{1}\{y_j\}$ denotes the one-hot encoding vector corresponding to the label of the sample x_j, and w_{uj} represents the cosine similarity between the feature representations $h_\theta(x_u)$ and $h_\theta(x_j)$. The memory bank M, as introduced in the previous section, stores the feature–label pairs used for reference during classification. The classifier defined by Eq. (8) can be interpreted as a similarity-weighted voting mechanism based on the memory bank. Specifically, for each test input x_u, the model retrieves the top-k most similar feature–label pairs from M, based on cosine similarity. It then computes a similarity-weighted sum for each label using the corresponding one-hot encodings and cosine similarity scores. The final predicted label is determined by selecting the one with the highest aggregated score after applying the softmax function. By leveraging feature representations and label information stored in the memory bank, this classification method improves prediction performance while significantly enhancing model interpretability. Compared to conventional machine learning or deep learning classifiers, it allows for a more transparent and analyzable decision-making process.

4.4.2. Test-time adaptation

Most existing test-time adaptation methods rely on unlabeled target domain samples to dynamically adjust the parameters of a pre-trained model, enabling it to better fit the distribution of the target domain. In contrast, AdaDoHTool achieves test-time adaptation by continuously updating a feature memory bank during the testing phase. This strategy allows the model to gradually align with the target domain distribution while effectively retaining the knowledge learned from the source domain, thereby ensuring compatibility between the two domains and improving both robustness and generalization performance.

At the beginning of the testing phase, the feature representations and corresponding labels of the source domain samples, extracted using the backbone network, are added to the feature memory bank $M = \left\{\left(h_\theta(x_i), y_i\right)\right\}_{i=1}^{n_s}$. During inference, it is assumed that each target sample x_u arrives in an online manner. The test-time adaptation process is summarized in Algorithm 1. For each incoming sample, the classifier generates a prediction. If the confidence of the predicted result $\hat{y}_u$ exceeds a predefined threshold, the sample's feature representation along with its predicted label is added to the memory bank $M \leftarrow M \cup \left(h_\theta(x_u), \hat{y}_u\right)$. This memory augmentation mechanism enables the model to adapt to the target distribution incrementally during testing. When combined with the KNN-based classifier, this strategy allows the model to update its decision boundaries without altering network parameters.

Algorithm 1: Online Memory Enhanced Test-Time Adaptation

Input: Pre-trained backbone h_θ; memory bank M; incoming target-domain samples $\mathcal{D}_t = \{x_u\}$; neighbor number K; threshold τ; memory capacity C

Output: Predicted labels

foreach *incoming target-domain sample* $x_u \in \mathcal{D}_t$ **do**
  $z_u \leftarrow h_\theta(x_u)$;
  foreach $(z_i, y_i) \in M$ **do**
    $a_i \leftarrow \mathrm{sim}(z_u, z_i) = \dfrac{z_u^\top z_i}{\|z_u\|_2 \|z_i\|_2}$;
  $\mathcal{N}_K(z_u) \leftarrow \mathrm{TopK}_{(z_i,y_i)\in\mathcal{M}}(a_i)$;
  foreach $c \in \mathcal{Y}$ **do**
    $s_c \leftarrow \sum_{(z_i,y_i)\in\mathcal{N}_K(z_u)} a_i \cdot \mathbb{I}(y_i = c)$;
  $p_K(x_u) \leftarrow \mathrm{softmax}\left(\{s_c\}_{c\in\mathcal{Y}}\right)$;
  $\hat{y}_u \leftarrow \arg\max_{c\in\mathcal{Y}} p_K^c(x_u)$;
  $\rho_u \leftarrow \max_{c\in\mathcal{Y}} \dfrac{\exp(s_c)}{\sum_{c'\in\mathcal{Y}} \exp(s_{c'})}$;
  if $\rho_u \geq \tau$ **then**
    $M \leftarrow M \cup \{(h_\theta(x_u), \hat{y}_u)\}$;
    if $|M| > C$ **then**
      Remove the oldest entry from M by FIFO;
  Output $\hat{y}_u$;

In addition, AdaDoHTool supports a pre-training and fine-tuning paradigm. Specifically, the model is first pretrained on the source domain dataset to obtain an initial model with strong transferability. Subsequently, a small number of labeled samples from the target domain are used to fine-tune the model, further adapting it to the target distribution. Under this training paradigm, the memory bank is updated in the same manner as during the source domain training. Feature–label pairs from the target domain used in fine-tuning are added to the memory bank following the same protocol, thereby enhancing the model's sensitivity to target-specific features and improving its cross-domain recognition performance without modifying the overall training procedure.

4.5. Complexity analysis

To further analyze the deployability of AdaDoHTool, we discuss the computational and memory complexity of the proposed memory-enhanced inference mechanism. Let C denote the capacity of the feature memory bank, d denote the dimension of the feature representation extracted by the backbone network, K denote the number of nearest neighbors, and $|\mathcal{Y}|$ denote the number of traffic classes.

For each incoming sample, AdaDoHTool first extracts its feature representation using the TextCNN backbone. Since the input sequence length and embedding dimension are fixed in our setting, the backbone inference cost for a single sample can be regarded as constant. Then, the KNN-based classifier computes the cosine similarity between the target feature vector and all feature vectors stored in the memory bank, which has a time complexity of $O(Cd)$.

After obtaining the similarity scores, the model selects the top-K nearest neighbors from the memory bank. If a heap-based selection strategy is used, this step has a time complexity of $O(C \log K)$. The similarity-weighted voting over the selected neighbors has a complexity of $O(K|\mathcal{Y}|)$. Therefore, the overall inference complexity for one test sample can be expressed as:

$$O(Cd + C\log K + K|\mathcal{Y}|). \tag{9}$$

Since d, K, and $|\mathcal{Y}|$ are fixed hyperparameters in our implementation, the inference cost of AdaDoHTool mainly grows linearly with the memory bank capacity C. For a batch of N test samples, the total inference complexity is:

$$O\big(N(Cd + C\log K + K|\mathcal{Y}|)\big). \tag{10}$$

The memory consumption mainly comes from storing feature–label pairs in the memory bank. Each entry contains a d-dimensional feature vector and its corresponding label. Therefore, the space complexity of the memory bank is:

$$O(Cd). \tag{11}$$

Overall, the time and space complexity of the memory-enhanced inference mechanism grow linearly with the memory bank capacity C. When the memory bank capacity is bounded, the proposed mechanism can maintain online adaptation while controlling computational and memory overhead.

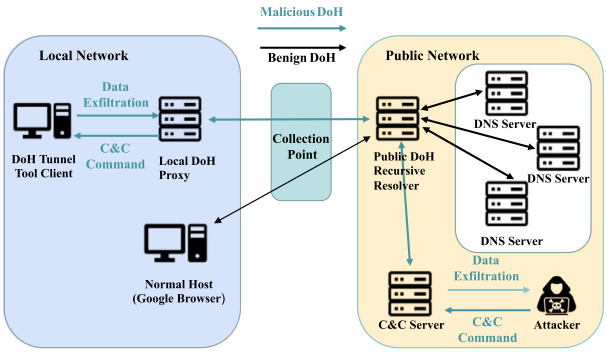


Fig. 4. Topology diagram of the traffic collection environment.

5. Performance evaluation

This section conducts experimental studies to evaluate the performance of our method. We first describe the experimental setting. Then, we analyze the feature distribution across different domains and compare AdaDoHTool with several baselines to assess its effectiveness and efficiency.

5.1. Experiment setting

Dataset Description. To enrich the experimental data, we extend the CIRA-CIC-DoHBrw-2020 dataset. The original dataset contains DoH traffic generated using four foreign public DoH recursive resolvers, namely AdGuard, Cloudflare, Google DNS, and Quad9. However, the number of covered resolver types and tunneling tools remains limited. To address this limitation, we further collect benign DoH traffic and malicious DoH tunneling traffic based on four additional public DoH resolvers, including the domestic resolvers DNSPod and Qihoo-360, as well as the foreign resolvers OpenDNS and Mullvad. Among them, the traffic collected from Qihoo-360, together with the resolver traffic from the CIRA-CIC-DoHBrw-2020 dataset, is used for the main experiments. In addition, we incorporate the DoH-Tunnel-Traffic-HKD dataset to supplement traffic generated by more DoH tunneling tools. The overall traffic collection topology is shown in Fig. 4.

For benign DoH traffic collection, we use Google Chrome, which supports the DoH protocol, to simulate normal web browsing behavior. The browser interacts with the automation script through Chrome WebDriver, and a Python script is used to automatically visit the top 200 most cited websites in The Majestic Million [30].

For malicious DoH tunneling traffic collection, we establish an experimental environment using two hosts. Host A deploys the server-side component of a DoH tunneling tool and acts as a disguised authoritative DNS server. Host B deploys the corresponding client-side component to

simulate a compromised endpoint. Host A remains continuously online, while Host B periodically establishes and terminates DoH tunneling sessions under the control of an automation script. Each session lasts for 5 min. For tunneling tools such as dns2tcp that do not directly support DoH, we use dnsproxy to forward conventional DNS traffic as DoH traffic. To avoid the impact of DoH connection reuse on sample independence, an independent dnsproxy instance is assigned to each newly launched tunneling client during traffic collection. The newly collected traffic was gathered over two weeks through multiple daytime collection rounds to reduce temporal bias. Each benign DoH collection round lasted approximately 8 h, while each malicious DoH tunneling collection round lasted approximately 10 h. The traffic collection point is deployed on the client side. During collection, we disable unrelated applications to reduce interference from background traffic.

Data Preprocessing. After collecting raw traffic, we extract unidirectional flows from the pcap files and retain only flows whose source or destination IP address matches the specified public DoH recursive resolver. For each retained flow, we parse TLS records and keep only TLS application data records. Their lengths are arranged in temporal order to construct a TLS application data record length sequence. Following our previous work, we use the first 15 TLS application data record lengths of each unidirectional flow as the input sequence, and pad shorter sequences with zeros. The resulting sequences are transformed into word embedding matrices and fed into the TextCNN backbone of AdaDoHTool.

To reduce data imbalance across resolvers, we sample 3000 benign DoH flows from each public resolver and 1000 malicious flows for each DoH tunneling tool. For categories with fewer available samples, all samples are retained. Section 5.2 further analyzes representative resolvers to illustrate their feature differences.

Baselines. We select nine representative models as baselines, including five classic domain adaptation algorithms (DSAN, DSAN-AT, DAN, DANN, and CDANs), two test-time adaptation algorithms (T3A and Tent), a hybrid deep learning framework for DoH tunnel detection (METC), and our previously developed model for DoH tunneling tool identification (WEDoHTool), as detailed below:

- **Deep Subdomain Adaptation Network (DSAN)** [21]: DSAN leverages the Local Maximum Mean Discrepancy (LMMD) to align subdomain distributions by minimizing domain-specific layer activations between the source and target domains, allowing effective transfer learning.
- **DSAN with Attention Mechanism (DSAN-AT)** [31]: Designed for encrypted IoT traffic classification, DSAN-AT extends DSAN by introducing both channel and spatial attention mechanisms.
- **Deep Adaptation Network (DAN)** [20]: DAN introduces Multi-Kernel Maximum Mean Discrepancy (MK-MMD) at fully connected layers to reduce distributional differences between source and target data, improving model transferability.
- **Domain-Adversarial Neural Network (DANN)** [22]: DANN incorporates adversarial training into transfer learning by introducing a domain classifier. The feature extractor learns domain-invariant and task-discriminative features by minimizing domain distinguishability.
- **Conditional Domain Adversarial Network (CDAN)** [23]: CDAN conditions the adversarial adaptation process on classifier predictions. It captures the cross-covariance between learned features and predicted labels via multilinear conditioning to improve discriminative alignment.
- **Test-Time Template Adjuster (T3A)** [25]: T3A improves generalization in unseen domains by computing pseudo-prototype representations for each class using unlabeled test-time data and a source-trained classifier.
- **Test Entropy Minimization via Feature Modulation (Tent)** [24]: Tent performs online adaptation during inference by minimizing the entropy of model predictions on target samples, enhancing robustness to domain shifts without requiring labeled data.

Table 2
Model hyperparameter settings.

Hyperparameter	Candidate values	Selection
Input Sequence Length	15	15
Embedding Dimension	64	64
Conv. Window Height	3, 4, 5	3, 4, 5
Max Pooling Height	13, 12, 11	13, 12, 11
Dropout Rate	0.1, 0.3, 0.5, 0.7	0.5
Optimizer	SGD, Adam, RMSProp	SGD
Batch Size	16, 32, 64, 128	32
k	25, 50, 75, 100, 125, 150, 200	25, 200
Learning Rate	0.1, 0.01, 0.001	0.01
Temperature	0.01, 0.05, 0.1, 0.25, 0.5	0.1
Memory Bank Capacity	30 000	30 000
N_{STEPS}	3001, 4001, 5001, 6001	5001
β	0.0, 0.25, 0.75, 1.1	0.75
Loss Function	Cross-Entropy, L_{KNN}	L_{KNN}

- **METC-Seq** [19]: METC is a CNN-BiGRU-Attention framework for encrypted DoH traffic detection and tunnel identification. Since the original METC uses flow-level statistical features, we implement a sequence-based variant, denoted as METC-Seq.
- **WeDoHTool** [6]: WEDoHTool is a two-stage framework for DoH tunneling tool identification, using a lightweight TextCNN for DoH traffic filtering and a Transformer encoder with a self-attention mechanism for tool identification.

To ensure a fair comparison, all baselines except DSAN-AT are implemented using the same TLS application data record length embeddings as AdaDoHTool. For METC, we adapt its original CNN-BiGRU-Attention architecture to the same sequence-based input and denote this variant as METC-Seq. This setting ensures that performance differences are mainly attributed to model design and adaptation mechanisms rather than different input features.

Hyperparameter Setting. Due to the randomness in sample selection, we randomly sample the data for each DoH public recursive resolver five times. For each experiment, the training and testing datasets are split using three different random seeds. The average performance across these runs is reported as the final result. Hyperparameters are selected from predefined candidate ranges through multiple experiments, as summarized in Table 2.

Environment Settings. The experimental environment is an Ubuntu 22.04 server equipped with an Intel(R) Core(TM) i9-14900 K CPU, Python language version 3.9, PyTorch-GPU 2.1.0, NVIDIA GeForce RTX 4090, and 32 GB of memory.

Methodology. We partition the dataset into five domains based on the DoH recursive resolvers: AdGuard, Cloudflare, Google DNS, Quad9, and Qihoo-360. Each domain contains four classes: benign-DoH, dns2tcp, dnscat2, and iodine. The experiments are divided into two major categories: Recognition performance on the CIRA-CIC-DoHBrw-2020 dataset, covering the four foreign DoH resolvers (Experiments $1-4$); Cross-region transfer performance from foreign resolvers to the domestic resolver (i.e., Experiment 5). The details of the five experimental setups are provided in Table 3. Furthermore, to simulate varying levels of supervision, we evaluate the model under three settings with 0%/10%/20% of labeled samples available in the target domain. The experiments are designed from five key perspectives: comparison with baseline methods, generalization capability evaluation, parameter sensitivity analysis, ablation study, and inference time and memory consumption analysis.

Performance Indicators. We comprehensively assess AdaDoHTool based on the following metrics:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}. \tag{12}$$

$$Recall = \frac{TP}{TP + FN}. \tag{13}$$

Table 3
Experiment group settings.

Experiment group	Source domain	Target domain
Experiment 1	Cloudflare Google DNS Quad9	AdGuard
Experiment 2	AdGuard Google DNS Quad9	Cloudflare
Experiment 3	AdGuard Cloudflare Quad9	Google DNS
Experiment 4	AdGuard Cloudflare Google DNS	Quad9
Experiment 5	AdGuard Cloudflare Quad9 Google DNS	Qihoo-360

$$FUR = \frac{N_{known \to unknown}}{N_{known}}. \tag{14}$$

$$Macro\text{-}F1 = \frac{1}{|\mathcal{Y}|} \sum_{c \in \mathcal{Y}} F1_c. \tag{15}$$

For a given class, TP represents the number of samples correctly predicted as this class, TN represents the number of samples correctly predicted as not belonging to this class, FP represents the number of samples incorrectly predicted as this class, and FN represents the number of samples belonging to this class but incorrectly predicted as other classes. In the open-set setting, N_{known} denotes the total number of known-class samples, and $N_{known \to unknown}$ represents the number of known-class samples incorrectly rejected as unknown. False Unknown Rate (FUR) measures the proportion of known-class samples that are incorrectly rejected as unknown. Macro-F1 averages the F1-scores over all classes and provides a balanced evaluation when class distributions are uneven.

5.2. Key observation on feature distribution

To more intuitively illustrate the differences in traffic features among DoH recursive resolvers, this section analyzes the traffic patterns of five resolvers from the constructed dataset. Prior work by Bannat Wala et al. [10] found that in server-to-client DoH communication, packet lengths often cluster at specific values. For instance, Cloudflare and Google DNS display distinct bimodal distributions, with peaks at 0–30 bytes and 490–530 bytes, respectively. In contrast, Quad9 exhibits a peak near 0 bytes and a quasi-normal distribution centered at approximately 170 bytes. These patterns suggest variations in padding strategies across resolvers.

From the perspective of TLS application data record lengths, Fig. 5 presents distributions for the five resolvers. The x-axis represents TLS record lengths, rounded to the nearest 10-byte bin, and the y-axis indicates the proportion of each length in the overall record length distribution. AdGuard's records span 0–500 bytes, peaking around 100–110 bytes without other prominent peaks. Quad9's lengths concentrate in 0–400 bytes, with peaks near 100 bytes and 260 bytes. Cloudflare and Google DNS share similar distributions, with peaks at 30–40 bytes and 100–110 bytes, indicating comparable strategies. Qihoo-360's distribution, also within 0–500 bytes, has multiple peaks at 30–60, 100, 160, and 290 bytes, implying a more complex approach.

Beyond analyzing TLS record length distributions, this section also explores the feature space distribution of word embeddings across resolvers. In Fig. 6, we apply UMAP to visualize the three-dimensional distribution of dnscat2 traffic features generated under different resolvers. The five colors in the figure represent the feature distributions

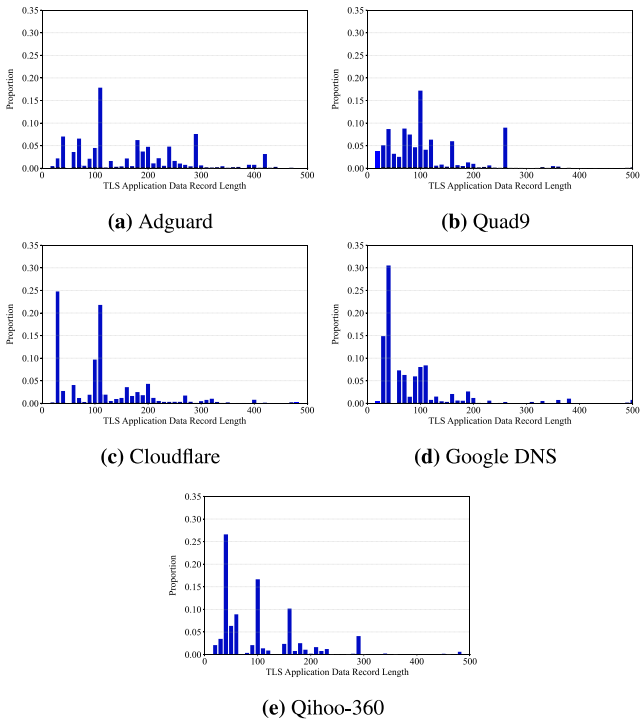


Fig. 5. TLS application record length distribution across different recursive resolvers.

of the dnscat2 category from five distinct domains. After dimensionality reduction, the dnscat2 traffic from most domains forms relatively independent clusters. Qihoo-360 is located in a relatively isolated region in the upper right corner, exhibiting a clear aggregation pattern. Cloudflare is mainly distributed in the upper left and lower right, partially overlapping with Quad9. The data points of Google DNS are concentrated on the left side, while AdGuard primarily appears in the upper half of the space. Both Google DNS and AdGuard show more dispersed distributions, with significant overlaps with other resolvers.

These observations are consistent with the previously discussed results on the TLS application record length distributions. The dnscat2 traffic features across different resolvers tend to form localized clusters, yet overlap at a broader scale, indicating that traffic of the same tunneling tool exhibits certain domain-specific differences in its feature representations.

Overall, TLS record length distributions confirm that DoH traffic characteristics differ among resolvers. The embedding visualizations further indicate that resolver configuration can alter the behavioral patterns and feature representations of identical DoH tunneling tools.

5.3. Comparison with baselines

We select nine representative models as baselines. Given that AdaDoHTool supports both training without labeled target domain samples and fine-tuning with partial target labels, we divide the comparative experiments into two scenarios: (1) Unlabeled target domain setting, where T3A, Tent, METC-Seq, and WEDoHTool are used as baselines; (2) Partially labeled target domain setting, where DSAN, DSAN-AT, DAN, DANN, CDANs, METC-Seq, and WEDoHTool are used as baselines.

Comparison under the unlabeled target domain setting. In this setting, T3A, Tent, METC-Seq, and WEDoHTool are used as baseline models. As shown in Table 4, AdaDoHTool achieves the highest average accuracy among all compared methods, with an average accuracy exceeding 0.70. Compared with T3A and Tent, AdaDoHTool obtains

Table 4
Statistical comparison of accuracy under the 0% labeled target-domain setting.

Method	Exp. 1		Exp. 2		Exp. 3		Exp. 4		Exp. 5		Avg.Acc	
	Mean ± Std	P-value	Mean ± Std	P-value	Mean ± Std	P-value	Mean ± Std	P-value	Mean ± Std	P-value	Mean ± Std	P-value
T3A	0.6028 ± 0.0358	0.042	**0.7725 ± 0.0177**	0.762	**0.7642 ± 0.0123**	0.875	0.7325 ± 0.0102	0.018	0.5593 ± 0.0044	0.064	0.6863 ± 0.0070	0.021
Tent	0.5850 ± 0.0396	0.004	0.7501 ± 0.0145	0.036	0.7233 ± 0.0159	0.041	0.6807 ± 0.0086	0.002	0.5815 ± 0.0066	0.486	0.6641 ± 0.0098	0.004
METC-Seq	0.5411 ± 0.0265	0.002	0.7519 ± 0.0186	0.043	0.7514 ± 0.0172	0.624	0.7235 ± 0.0138	0.011	0.5521 ± 0.0097	0.026	0.6640 ± 0.0104	0.004
WEDoHTool	0.5341 ± 0.0248	0.001	0.6447 ± 0.0215	0.001	0.7276 ± 0.0196	0.073	0.6997 ± 0.0158	0.004	**0.5888 ± 0.0112**	0.671	0.6390 ± 0.0121	0.002
AdaDoHTool	**0.6988 ± 0.0822**	–	0.7711 ± 0.0280	–	0.7443 ± 0.0255	–	**0.7661 ± 0.0281**	–	0.5825 ± 0.0228	–	**0.7125 ± 0.0171**	–

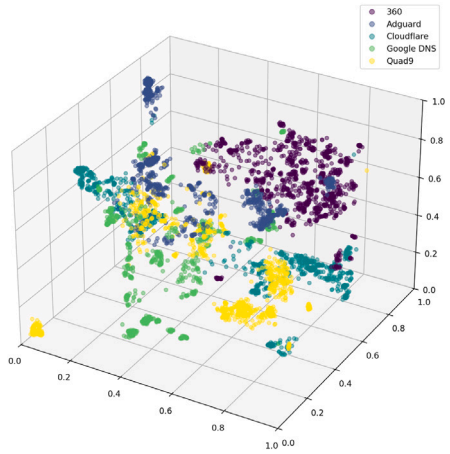


Fig. 6. The UMAP visualization of feature vectors for dnscat2 traffic across different recursive resolvers (domains).

better overall performance, demonstrating the superiority of the proposed memory-enhanced adaptation mechanism over other TTA methods in the unsupervised setting. Compared with METC-Seq and WEDoHTool, AdaDoHTool also achieves higher average accuracy, indicating that introducing test-time adaptation helps mitigate resolver-induced distribution shifts.

We further observe that AdaDoHTool is slightly outperformed by specific baselines in a few individual experiments. In particular, T3A achieves slightly higher accuracy in Exp. 2 and Exp. 3. As discussed in Section 5.2, Cloudflare and Google DNS exhibit similar TLS application data record length distributions, with common peaks around 30–40 bytes and 100–110 bytes. Since T3A adjusts the classifier at test time by constructing class-wise pseudo-prototypes from unlabeled target samples, such distributional similarity may help it obtain more reliable class-level decision references in these two experiments. However, this advantage is not consistent across all experiments. T3A performs worse than AdaDoHTool in Exp. 1, Exp. 4, and Exp. 5, indicating that its advantage is sensitive to the source-target resolver combination. This is because T3A performs adaptation by estimating class-wise pseudo-prototypes from unlabeled target samples. When the target resolver induces larger traffic distribution changes, these estimated prototypes may become less reliable, thereby weakening its test-time adjustment. In contrast, AdaDoHTool maintains a feature–label memory bank and performs similarity-weighted KNN voting over sample-level representations, enabling it to exploit target-domain information in a more fine-grained manner during inference.

In Exp. 5, AdaDoHTool achieves an accuracy of 0.5825, which is slightly lower than that of WEDoHTool, but the performance gap is small. WEDoHTool is a word-embedding-based method for DoH tunneling tool identification and also benefits from TLS record length representations. Since WEDoHTool does not update its decision mechanism during testing, it is not affected by potentially unreliable pseudo-label updates under this specific target resolver. As discussed in Section 5.2, Qihoo-360 exhibits a more complex TLS application data record length distribution, with several visible peaks around 30–60, 100, 160, and 290 bytes, whereas the source resolvers show different peak patterns. Under such a stronger resolver-induced distribution shift, the high-confidence memory update in AdaDoHTool may provide limited adaptation gain when reliable target-domain evidence is insufficient. This may explain why WEDoHTool performs slightly better in Exp. 5. However, AdaDoHTool still outperforms WEDoHTool in the first four experiments and achieves an average accuracy approximately 11.50% higher, indicating the robustness of the proposed memory-enhanced adaptation mechanism across different unknown-resolver settings.

To further verify whether the observed performance improvements are statistically meaningful, we also report the standard deviation and the results of the one-sided paired Wilcoxon signed-rank test in Table 4. The statistical results show that, under the 0% labeled target-domain setting, AdaDoHTool achieves statistically significant improvements in average accuracy over the baselines, indicating that its overall performance gain is not merely caused by random sampling or random data splitting.

Overall, although AdaDoHTool is slightly outperformed by specific baselines in a few cases, it still achieves the highest average accuracy. These results demonstrate that AdaDoHTool provides stronger overall robustness under the unlabeled target-domain setting, while also indicating that adaptation under severe resolver-induced distribution shifts remains challenging.

Comparison under the partially labeled target domain setting. In this setting, DSAN, DSAN-AT, DAN, DANN, and CDANs are used as domain adaptation baselines. Table 5 summarizes the performance under the scenarios with 10% and 20% labeled target-domain samples. Compared with the unlabeled setting, introducing a small proportion of labeled target-domain data leads to significant performance improvements across all experimental groups. These labeled target samples effectively mitigate the contamination caused by initial label noise in the feature memory bank, thereby enhancing the model's transferability and classification performance.

When 10% labeled target-domain samples are available, AdaDoHTool achieves an average accuracy of 0.8463. Compared with the 0% labeled setting, this result shows that a small number of labeled target-domain samples can further improve the performance of AdaDoHTool. This is because labeled target samples provide more reliable target-domain feature–label pairs for the memory bank, reducing the influence of noisy pseudo-labels during inference. Compared with WEDoHTool and METC-Seq, AdaDoHTool improves the average accuracy by 17.51% and 13.95%, respectively. In addition, AdaDoHTool outperforms all domain adaptation baselines. Compared with DAN, the best-performing DA baseline, AdaDoHTool improves the average accuracy by 10.27%, further validating the effectiveness of the proposed method. Exp. 5 remains the most challenging setting because Qihoo-360 exhibits a TLS application data record length distribution that differs from those of the source resolvers. Nevertheless, AdaDoHTool still achieves the best performance in this experiment, improving the accuracy from 0.5825 in the 0% labeled setting to 0.6944 in the 10% labeled setting. Although the improvement margin is smaller than that in the other experiments, this result shows that limited labeled target-domain samples still provide useful reference information for the memory bank under stronger resolver-induced distribution shifts.

Table 5 also reports the results obtained with 20% labeled target-domain samples. Compared with the unlabeled setting, 20% labeled

Table 5
Comparison of model performance under 10% and 20% labeled target domain samples.

Method	10% labeled target samples						20% labeled target samples					
	Exp. 1	Exp. 2	Exp. 3	Exp. 4	Exp. 5	Avg.Acc	Exp. 1	Exp. 2	Exp. 3	Exp. 4	Exp. 5	Avg.Acc
DSAN	0.5421	0.6745	0.7817	0.7271	0.5179	0.6486	0.4921	0.6743	0.7752	0.7379	0.5173	0.6394
DSAN-AT	0.5556	0.2642	0.4626	0.5004	0.4444	0.4454	0.5609	0.2642	0.4652	0.4996	0.3750	0.4330
DAN	0.8166	0.7795	0.8284	0.8361	0.5769	0.7675	0.7705	0.7726	0.8182	0.8526	0.5750	0.7578
DANN	0.6995	0.6787	0.8278	0.8396	0.4283	0.6948	0.6417	0.6194	0.8421	0.8315	0.4716	0.6813
CDANs	0.8175	0.6212	0.8222	0.8005	0.4233	0.6969	0.7225	0.6401	0.8167	0.8174	0.3548	0.6703
METC-Seq	0.8252	0.7182	0.7756	0.7610	0.6334	0.7427	0.8371	0.7625	0.7786	0.7447	0.6431	0.7532
WEDoHTool	0.8050	0.6748	0.7585	0.7187	0.6438	0.7202	0.8208	0.7769	0.7582	0.7428	0.6658	0.7529
AdaDoHTool	**0.8732**	**0.8678**	**0.8797**	**0.9164**	**0.6944**	**0.8463**	**0.8742**	**0.8186**	**0.8639**	**0.9145**	**0.6905**	**0.8323**

samples still bring clear performance gains, confirming the usefulness of labeled target-domain information. Compared with the 10% setting, the performance does not always increase monotonically. This may be because more labeled samples do not necessarily provide better coverage of the target-domain distribution. Due to intra-class variations across different DoH resolvers and the uneven target-domain feature space, additional samples may be concentrated in high-density regions while still failing to cover boundary areas or sparse sub-clusters, thus providing limited benefit for decision-boundary adjustment. WEDoHTool benefits more from additional labeled target samples because it does not perform test-time adaptation and thus relies more on labeled target data to compensate for resolver-induced distribution shifts. In contrast, AdaDoHTool already achieves strong performance with only 10% labeled target samples by incorporating reliable target-domain feature–label pairs into the memory bank, so increasing the labeled ratio to 20% provides limited additional benefit in some experiments.

Overall, AdaDoHTool consistently outperforms all baseline models under both 10% and 20% labeled target-domain settings. The results demonstrate the robustness and effectiveness of AdaDoHTool in handling distribution shifts caused by diverse DoH resolvers.

5.4. Generalization evaluation

To further evaluate the generalization capability of AdaDoHTool, this section conducts additional experiments from two perspectives. First, we introduce more previously unseen DoH recursive resolvers to evaluate the identification performance of the model under resolver-induced distribution shifts. Second, we introduce DoH tunneling tools that are not observed during training to assess the open-set recognition capability of the model when facing emerging tunneling tools.

5.4.1. Evaluation on unknown resolvers

To evaluate the adaptability of AdaDoHTool to additional unknown resolvers, we collect traffic from three public DoH resolvers, including the domestic resolver DNSPod and two foreign resolvers, OpenDNS and Mullvad. For Mullvad, we use its ad-blocking version. This experiment follows the same closed-set classification setting as the previous experiments, where the label space consists of benign-DoH, dns2tcp, dnscat2, and iodine.

To cover different types of resolver-induced distribution shifts, we design four additional experimental settings, denoted as Exp. 6–Exp. 9. In Exp. 6, OpenDNS is used as the unseen target domain, while AdGuard, Cloudflare, Google DNS, and Quad9 are used as source domains. This setting evaluates the generalization capability of the model to an additional foreign resolver. In Exp. 7, DNSPod is used as the unseen target domain, with the same four foreign resolvers as source domains. This setting evaluates cross-region transfer from foreign resolvers to a domestic resolver. In Exp. 8, the Mullvad ad-blocking resolver is used as the unseen target domain, again using the four foreign resolvers as source domains. This setting examines the influence of a resolver with a special service configuration on model performance. In Exp. 9, Qihoo-360 is further added to the source domains, and DNSPod is used as the target domain. This setting is designed to analyze whether increasing source-domain diversity with domestic resolver samples can mitigate cross-region distribution shifts and improve performance on a challenging domestic target resolver. Consistent with the previous experiments, we consider two supervision settings with 0% and 10% labeled target-domain samples.

Table 6
Generalization performance on additional unknown resolvers.

Labels	Method	Exp. 6	Exp. 7	Exp. 8	Exp. 9
0%	T3A	**0.7737**	0.5838	0.6521	0.6037
	Tent	0.7338	0.6049	0.6335	0.6429
	AdaDoHTool	0.7723	**0.6095**	**0.7304**	**0.6892**
10%	DSAN	0.6318	0.5225	0.5475	0.6037
	DSAN-AT	0.4639	0.4459	0.4671	0.4850
	DAN	0.7114	0.5442	0.6178	0.5771
	DANN	0.6824	0.4847	0.5903	0.4538
	CDANs	0.8140	0.4995	0.6266	0.5515
	METC-Seq	0.7893	0.6621	0.7525	0.7016
	WEDoHTool	0.7970	0.6553	0.7396	0.6848
	AdaDoHTool	**0.8958**	**0.7426**	**0.8143**	**0.7887**

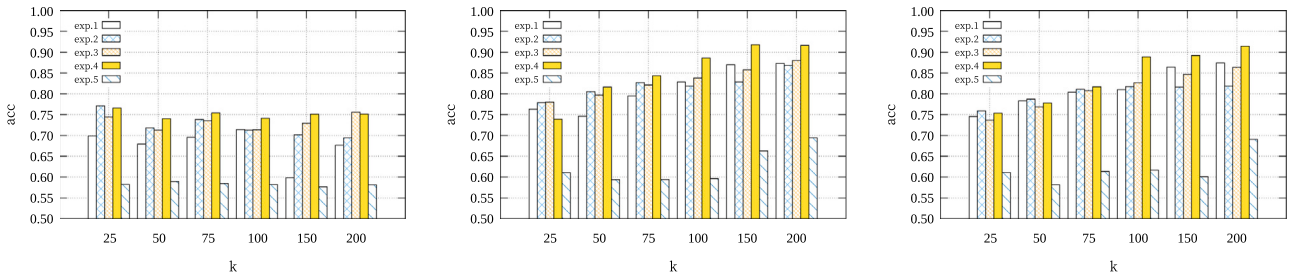
The detailed results of the four additional resolver settings are reported in Table 6. The results show that AdaDoHTool maintains good generalization capability on additional unseen resolvers. Under the 0% labeled target setting, AdaDoHTool outperforms Tent on average and achieves better performance than T3A in all settings except Exp. 6. In Exp. 6, where OpenDNS is used as the additional foreign target resolver, AdaDoHTool achieves performance comparable to that observed on the original foreign resolvers in Exp. 1–Exp. 4. This result provides further evidence of AdaDoHTool's cross-resolver generalization capability beyond the original resolver set.

In contrast, when DNSPod is used as the target domain in Exp. 7, all methods experience performance degradation. AdaDoHTool achieves an accuracy of 0.6095 under the 0% labeled setting, which is notably lower than 0.7723 in Exp. 6. This suggests that domestic resolvers introduce stronger distribution shifts. In Exp. 8, using the Mullvad ad-blocking resolver as the target domain leads to an accuracy of 0.7304, also lower than the OpenDNS setting, indicating that special resolver configurations can also affect traffic distributions.

Under the 10% labeled target setting, AdaDoHTool achieves the best performance in all four experiments, with accuracies of 0.8958, 0.7426, 0.8143, and 0.7887, respectively. This demonstrates that a small amount of labeled target data helps the memory bank absorb target-domain feature information. Comparing Exp. 7 and Exp. 9 further shows that adding Qihoo-360 to the source domains improves the performance on DNSPod from 0.6095 to 0.6892 under the 0% setting, and from 0.7426 to 0.7887 under the 10% setting. These results indicate that incorporating resolver samples closer to the target domain can mitigate performance degradation in cross-region transfer. Overall, AdaDoHTool remains stable on additional resolvers, although resolver differences still introduce non-negligible distribution shifts.

5.4.2. Evaluation on unseen tunneling tools

To evaluate the robustness of AdaDoHTool against emerging tunneling tools, we conduct an open-set recognition experiment. During training, only four known classes are used, including benign-DoH,



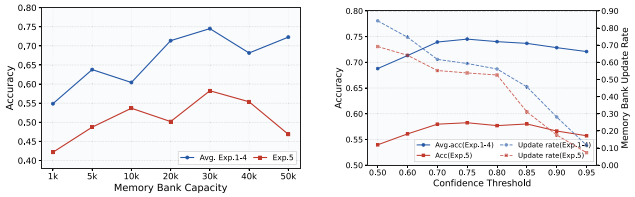
(a) 0% labeled target samples **(b)** 10% labeled target samples **(c)** 20% labeled target samples

Fig. 7. Performance under different k in 0%, 10%, and 20% labeled target domain settings.

Table 7
Open-set evaluation on unseen DoH tunneling tools.

Method	K-Acc	U-Rec	FUR	Macro-F1
T3A	0.8593	0.3933	0.1025	0.6804
Tent	0.8936	0.6573	0.1049	0.8063
METC-Seq	0.8927	0.6269	0.1055	0.7954
WEDoHTool	0.8975	**0.6598**	0.1010	0.8101
AdaDoHTool	**0.9337**	0.6253	**0.0658**	**0.8429**

Note: K-Acc denotes known-class accuracy and U-Rec denotes unknown recall.



(a) Memory bank capacity **(b)** Confidence threshold

Fig. 8. Sensitivity analysis of memory bank capacity and confidence threshold.

dns2tcp, dnscat2, and iodine. The three additional tools, dnstt, tcp-over-dns, and tuns, are excluded from training and appear only during testing, where they are treated as the unknown class.

The source domain is constructed by mixing traffic from four public resolvers, namely AdGuard, Google DNS, Cloudflare, and Quad9, while the target data are from DoH-Tunnel-Traffic-HKD. Unlike the strict leave-one-resolver-out setting used in the previous experiments, this experiment does not emphasize strong resolver-level domain transfer. Instead, it focuses on whether the model can identify tunneling tools that are unseen during training under a relatively consistent resolver distribution. Therefore, the known-class recognition performance in this experiment is generally higher than that in the previous 0% labeled unknown-resolver setting. We select WEDoHTool, T3A, Tent, and METC as baseline methods. The results are reported in Table 7. We compute the confidence scores of each method only on a held-out validation set from known classes, and use the lower 10th percentile as the rejection threshold.

The results show that AdaDoHTool achieves the highest Known Accuracy, the lowest FUR, and the highest Macro-F1. This indicates that AdaDoHTool better preserves known-class recognition performance while reducing the false rejection of known samples as unknown. Although WEDoHTool and Tent obtain slightly higher Unknown Recall, their higher FUR and lower Known Accuracy indicate a more aggressive rejection behavior, which improves unknown-tool detection at the cost of misclassifying more known samples as unknown. In contrast, the KNN consistency score of AdaDoHTool relies on neighborhood consistency in the memory bank, leading to a more stable rejection strategy and a better overall balance among Known Acc, FUR, and Macro-F1. Overall, open-set recognition of unseen tunneling tools remains challenging, but AdaDoHTool shows a more balanced trade-off between known-class recognition and unknown-tool rejection.

5.5. Parameter sensitivity analysis

To evaluate the influence of key hyperparameters on AdaDoHTool, this section conducts sensitivity analyses on the neighbor number k, the memory bank capacity, and the confidence threshold for memory updating. The analysis of k is performed on Exp. 1–Exp. 5 under 0%, 10%, and 20% labeled target-domain settings. The analyses of memory bank capacity and confidence threshold are conducted under the 0% labeled target-domain setting, which represents a stricter scenario because no labeled target-domain samples are available to guide adaptation. Their results are reported using two curves: the average accuracy over Exp. 1–Exp. 4 and the accuracy of Exp. 5. Unless otherwise specified, all other hyperparameters are kept the same as the default settings in Table 2.

Neighbor Number k. In experiments involving 0%, 10%, and 20% labeled target-domain samples, we evaluate the performance of AdaDoHTool under different neighbor numbers k, including 25, 50, 75, 100, 150, and 200.

As shown in Fig. 7(a), when no labeled target-domain samples are available, AdaDoHTool generally achieves better performance with smaller k values. As k increases, the accuracy tends to fluctuate and decline. This is because, in the fully unlabeled setting, the memory bank is updated only with pseudo-labeled target samples. A larger neighborhood may include more samples from confusing classes or incorrectly pseudo-labeled target samples, which can disturb the KNN voting process. In contrast, a smaller k makes the prediction rely on the most similar local neighbors, reducing the influence of noisy or irrelevant samples.

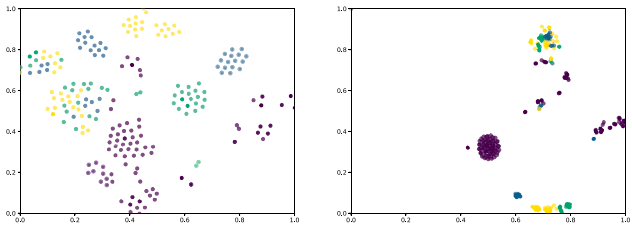
Fig. 7(b) and 7(c) show the results under 10% and 20% labeled target-domain settings. When a small number of labeled target samples are available, AdaDoHTool usually performs better with larger k values. This is because labeled target samples provide more reliable feature–label pairs in the memory bank and reduce the negative impact of pseudo-label noise. Under this condition, a larger neighborhood can aggregate more stable class evidence, smooth local fluctuations, and improve classification robustness.

Overall, the optimal choice of k is closely related to the proportion of labeled target samples, the reliability of the memory bank, the sample distribution, and the domain divergence. A smaller k is more suitable when the memory bank contains more uncertain pseudo-labeled samples, while a larger k is more effective when reliable labeled target samples are available. Accordingly, we set $k = 25$ for the 0% labeled target-domain setting and $k = 200$ for the 10% and 20% labeled target-domain settings in the following experiments.

Memory Bank Capacity. As shown in Fig. 8(a), the performance of AdaDoHTool is sensitive to the memory bank capacity. When the capacity is small, the memory bank cannot sufficiently cover the feature distributions of both source and target domains, resulting in unstable nearest-neighbor retrieval and degraded accuracy. As the capacity increases, the overall performance improves because more representative

Table 8
Ablation study of AdaDoHTool under the 0% labeled target-domain setting.

Method	Exp. 1	Exp. 2	Exp. 3	Exp. 4	Exp. 5	Avg.Acc
AdaDoHTool	0.6988	**0.7711**	**0.7443**	**0.7661**	0.5825	**0.7125**
w/o L_{KNN}	**0.7009** ▲0.30%	0.7549 ▼2.10%	0.7276 ▼2.24%	0.7587 ▼0.97%	**0.6021** ▲3.36%	0.7088 ▼0.52%
w/o memory update	0.5148 ▼26.33%	0.6286 ▼18.48%	0.6039 ▼18.86%	0.6612 ▼13.69%	0.5347 ▼8.21%	0.5886 ▼17.39%
w/o confidence threshold	0.5456 ▼21.92%	0.6660 ▼13.63%	0.6419 ▼13.76%	0.6914 ▼9.75%	0.5621 ▼3.50%	0.6214 ▼12.79%
w/o replacement	0.6333 ▼9.37%	0.6576 ▼14.72%	0.6353 ▼14.64%	0.6894 ▼10.01%	0.5627 ▼3.40%	0.6357 ▼10.78%
LRU replacement	0.6754 ▼3.35%	0.7312 ▼5.17%	0.7076 ▼4.93%	0.7358 ▼3.96%	0.5682 ▼2.45%	0.6836 ▼4.06%



(a) L_{KNN} loss function **(b)** Cross-entropy loss function

Fig. 9. Visualization of feature distributions with different loss functions.

feature–label pairs are preserved. However, the improvement is not strictly monotonic. A larger memory bank may also retain outdated samples or noisy pseudo-labeled target samples, which can disturb the local neighborhood structure used by KNN-based classification. The best overall performance is achieved when the capacity is set to 30,000. Therefore, we use 30,000 as the default memory bank capacity.

Confidence Threshold. As shown in Fig. 8(b), the confidence threshold controls the trade-off between adaptation ability and memory reliability. When the threshold is low, more target-domain samples are accepted into the memory bank, which increases the update rate but may also introduce noisy pseudo-labels and degrade performance. As the threshold increases, low-confidence samples are gradually filtered out, and the accuracy improves. The model achieves relatively good performance when the threshold is between 0.70 and 0.85, indicating that AdaDoHTool is reasonably stable within this range. However, when the threshold becomes too high, the update rate drops sharply, and insufficient target-domain samples are incorporated into the memory bank, limiting test-time adaptation. In particular, when the threshold is set to 0.75, AdaDoHTool achieves the best overall accuracy while maintaining a relatively high update rate. Therefore, we set the confidence threshold to 0.75 as a balanced choice between reliable memory updating and sufficient target-domain adaptation.

5.6. Ablation study

To evaluate the contribution of each key component in AdaDoHTool, we conduct ablation experiments under the 0% labeled target-domain setting. Table 8 reports the results of the full AdaDoHTool and five variants. The first variant, w/o L_{KNN}, replaces the proposed KNN-aware loss with the conventional cross-entropy loss. The second variant, w/o memory update, keeps the memory bank fixed during testing and does not insert target-domain samples. The third variant, w/o confidence threshold, inserts all target-domain samples into the memory bank regardless of prediction confidence. The fourth variant, w/o replacement, stops inserting new samples once the memory bank reaches its capacity. The fifth variant replaces the default FIFO replacement strategy with LRU, which removes the least recently retrieved samples. All other hyperparameters remain the same as the default settings.

First, we analyze the effect of the KNN-aware loss function. As shown in Table 8, both L_{KNN} and cross-entropy achieve competitive performance, indicating that the TextCNN backbone can learn discriminative representations with either training objective. Compared with cross-entropy, L_{KNN} achieves a higher average accuracy of 0.7125 versus 0.7088. Although cross-entropy performs slightly better in Exp. 1 and Exp. 5, L_{KNN} achieves higher accuracy in three out of five experiments and provides better overall performance. Nevertheless, L_{KNN} remains more suitable for AdaDoHTool because it is better aligned with the memory-bank-based KNN inference procedure. Unlike cross-entropy, which mainly optimizes a parametric classification objective during source-domain training, L_{KNN} explicitly optimizes the feature space with respect to nearest-neighbor retrieval. As illustrated in Fig. 9, the features learned with L_{KNN} show clearer inter-class separation while preserving richer intra-class structures. In contrast, cross-entropy produces more compact feature clusters, which may reduce the model's ability to adapt to resolver-induced distribution shifts. Therefore, the ablation results show that L_{KNN} provides a design-consistent representation learning objective for AdaDoHTool and leads to better average performance.

Second, we evaluate the effect of the memory update mechanism. When the memory bank is fixed during testing, the average accuracy drops from 0.7125 to 0.5886. This result shows that source-domain feature–label pairs alone are insufficient for unknown resolvers. By inserting high-confidence target-domain samples, AdaDoHTool gradually incorporates target-domain information and adjusts the effective decision boundary without updating the backbone parameters.

Third, we analyze the effect of the confidence threshold. Without this threshold, all target-domain samples are inserted into the memory bank, reducing the average accuracy to 0.6214. This indicates that confidence-based filtering is important for maintaining reliable memory-bank updates and stable KNN retrieval. The confidence threshold helps filter unreliable samples and improves the stability of online adaptation.

Finally, we evaluate the memory replacement strategy. The full AdaDoHTool adopts FIFO as the default policy. The w/o replacement variant reduces the average accuracy to 0.6357, showing that stopping updates after the memory bank is full limits adaptation to the target domain. Replacing FIFO with LRU performs better than w/o replacement but still lower than the default FIFO strategy, indicating that FIFO is a simple and effective replacement policy.

Overall, the ablation results verify the necessity of the main components in AdaDoHTool. The KNN-aware loss improves feature transferability, the memory update mechanism enables test-time adaptation, the confidence threshold reduces pseudo-label noise, and FIFO maintains an effective memory structure for evolving target-domain traffic.

5.7. Inference time and memory

To assess how increasing the feature memory bank's sample volume affects inference efficiency, we measure the average per-sample inference time for AdaDoHTool and all baselines on the study dataset. As shown in Table 9, AdaDoHTool maintains consistently low latency, with all cases under 0.03 ms, showing high computational efficiency.

In contrast, DSAN-AT incurs a significantly higher average inference time of 121.11 ms, far exceeding that of the other models. This increase primarily results from its deeper ResNet-based architecture and integrated attention mechanisms. Additionally, baseline models using adversarial training strategies (e.g., DANN, CDANs) tend to exhibit higher inference latency than distribution matching methods (e.g., DSAN), as

Table 9
Inference time per sample for different methods (ms).

Method	Inference time (ms)
DSAN	0.0214
DSAN-AT	121.1100
DAN	0.0235
DANN	0.0250
CDANs	0.0233
T3A	0.0177
Tent	0.0255
METC-Seq	0.0294
WEDoHTool	0.0356
AdaDoHTool	**0.0054**

the former involves more complex transformations and discrimination processes during inference, while the latter relies on simpler, fixed mappings. Notably, despite utilizing a dynamically updated memory bank and a KNN-based classifier, AdaDoHTool's average inference time is approximately one-third of that of T3A, the most efficient baseline method in terms of inference speed, further highlighting its runtime advantage.

For memory usage, the maximum bank size of 30,000 samples (covering the dataset) consumes only about 0.01095 GB (30000 ×(96 × 4 + 1 × 8)), imposing minimal overhead on modern CPUs and RAM. Overall, AdaDoHTool achieves superior recognition performance with high inference efficiency and low memory cost.

6. Discussion and future work

Despite the promising results achieved by AdaDoHTool in identifying DoH tunneling tools with unknown recursive resolvers, several issues still require further investigation.

Memory bank update strategies. The current implementation utilizes a fixed-size feature memory bank. The impact of different memory update strategies on model performance has not been fully explored. In future research, inspired by recent advances in continual learning [32], we will explore the following strategies: the Least Recently Used (LRU) strategy, which removes the least recently accessed samples; the confidence-based replacement strategy [33], which prioritizes retaining high-confidence samples while removing low-confidence ones; and entropy-balanced reservoir sampling [34], which maintains balanced and informative samples by optimizing the memory entropy, effectively mitigating bias caused by class imbalance in streaming data. Our additional streaming validation shows that the default FIFO strategy achieves the best overall performance, while the confidence-based replacement strategy performs better in the most challenging cross-region setting. These results indicate that memory update strategies can affect online adaptation and that different strategies may be suitable for different deployment scenarios. In future work, we will further investigate more advanced memory management strategies under larger-scale streaming [35].

Defense against noisy or mislabeled data. The method incorporates a feature memory bank and a dynamic update mechanism. By setting a confidence threshold, it determines whether new samples should be added to the memory bank. However, it cannot entirely prevent the inclusion of noisy data. The confidence threshold ensures that the model's predictions for certain samples are "high-confidence", but it does not guarantee that these samples are error-free. This pseudo-labeling strategy may cause the model to reinforce incorrect patterns over time [36]. Therefore, there is a need to design a contamination defense mechanism for the feature memory bank. A potential direction is noise detection and removal. We plan to introduce noise detection algorithms based on sample confidence, anomaly detection, or consistency validation mechanisms to filter out unreliable samples, thereby maintaining the purity of the memory bank. For example, by evaluating the model's output confidence and prediction consistency, we can dynamically assess whether new samples should be added to the memory bank. Another approach is to focus more on the representativeness and diversity of the samples, rather than solely relying on confidence. We plan to assess the contribution of new samples to the existing memory bank using clustering methods or distance metrics, prioritizing samples that are highly representative of the data distribution.

Robustness against adversarial attacks. The TLS application record length sequence used in this paper as a key feature, while effective in identifying DoH tunneling tool traffic, is also susceptible to adversarial attacks [37]. Existing DoH traffic obfuscation techniques [38] have been proposed to alter the TLS record lengths via padding strategies to evade traffic recognition, which means that malicious users can manipulate their traffic patterns through adversarial techniques to bypass current detection methods. With the rapid development of Generative Adversarial Networks (GANs) [39,40] and Diffusion Models [41,42], generative algorithms can increasingly produce adversarial samples that closely resemble the original traffic, further complicating adversarial attacks. Therefore, extending and improving the method proposed in this paper to defend against these new adversarial threats remains an urgent research challenge. We plan to integrate adversarial training methods, utilizing techniques such as GANs to generate adversarial DoH traffic samples and incorporate them into the training data, thereby enhancing the model's robustness.

7. Conclusion

In this paper, we propose AdaDoHTool, a test-time adaptive method for identifying DoH tunneling tools with unknown recursive resolvers. AdaDoHTool leverages a word embedding matrix generated from TLS application data record length sequences as input. A TextCNN-based backbone combined with a KNN-specific loss function is employed to learn transferable and discriminative feature representations. Furthermore, a feature–label memory bank with an online update mechanism and a KNN-based classifier enables continuous adaptation during inference, allowing the model to dynamically align with evolving traffic distributions without retraining. Experimental results demonstrate that AdaDoHTool achieves superior accuracy compared to baseline methods under both zero-label and few-label target domain conditions, with low memory consumption and fast inference. For future work, we will further investigate advanced strategies for dynamic memory updating, enhance robustness against noisy samples, and strengthen resistance to adversarial or obfuscated traffic patterns.

CRediT authorship contribution statement

Yandu Yi: Writing – original draft, Validation, Methodology, Investigation. **Xiaoyan Hu:** Writing – review & editing, Supervision, Resources, Formal analysis, Conceptualization. **Yang Miao:** Data curation, Conceptualization. **Guang Cheng:** Resources, Project administration. **Ruidong Li:** Supervision, Resources. **Hua Wu:** Resources, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] P. Hoffman, P. McManus, Dns queries over https (doh). Internet Requests for Comments, Technical Report, IETF, RFC, 8484, 2018.

[2] C. Patsakis, F. Casino, V. Katos, Encrypted and covert DNS queries for botnets: Challenges and countermeasures, Comput. Secur. 88 (2020) 101614.

[3] A. Turing, G. Ye, An analysis of godlua backdoor, 2019, https://blog.netlab.360.com/an-analysis-of-godlua-backdoor-en/. (Accessed: 23 September 2025).

[4] D. Vekshin, K. Hynek, T. Cejka, Doh insight: Detecting dns over https by machine learning, in: Proceedings of the 15th International Conference on Availability, Reliability and Security, 2020, pp. 1–8.

[5] T. Zebin, S. Rezvy, Y. Luo, An explainable AI-based intrusion detection system for DNS over https (DoH) attacks, IEEE Trans. Inf. Forensics Secur. 17 (2022) 2339–2349.

[6] Y. Miao, X. Hu, G. Cheng, R. Li, H. Wu, Y. Meng, WeDoHTool: Word embedding based early identification of DoH tunnel tool traffic in dynamic network environments, Comput. Secur. (2025) 104680.

[7] A. Mayrhofer, RFC 7830: The edns (0) padding option, 2016.

[8] A. Mayrhofer, Padding Policies for Extension Mechanisms for DNS (EDNS (0)),” Internet Requests for Comments, Technical Report, 2018, RFC Editor, RFC 8467, 2018.[Online]. Available: http://www.rfc-editor.org.

[9] J. Wu, Y. Zhu, B. Li, Q. Liu, B. Fang, Peek inside the encrypted world: Autoencoder-based detection of doh resolvers, in: 2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), IEEE, 2021, pp. 783–790.

[10] F. Bannat Wala, S. Campbell, M. Kiran, Insights into doh: Traffic classification for dns over https in an encrypted network, in: Proceedings of the 2023 on Systems and Network Telemetry and Analytics, 2023, pp. 9–17.

[11] K. Jerabek, K. Hynek, O. Rysavy, Comparative analysis of DNS over HTTPS detectors, Comput. Netw. 247 (2024) 110452.

[12] CIC, CIRA-CIC-DoHBrw-2020, 2020, https://www.unb.ca/cic/datasets/dohbrw-2020.html. (Accessed: 22 September 2025).

[13] M. MontazeriShatoori, L. Davidson, G. Kaur, A.H. Lashkari, Detection of doh tunnels using time-series classification of encrypted traffic, in: 2020 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, IEEE, 2020, pp. 63–70.

[14] T.A. Nguyen, M. Park, Doh tunneling detection system for enterprise network using deep learning technique, Appl. Sci. 12 (5) (2022) 2416.

[15] BCCC, Cicflowmeter, 2018, https://github.com/ahlashkari/CICFlowMeter. (Accessed: 23 September 2025).

[16] M. Zhan, Y. Li, G. Yu, B. Li, W. Wang, Detecting DNS over HTTPS based data exfiltration, Comput. Netw. 209 (2022) 108919.

[17] M. Moure-Garrido, C. Campo, C. Garcia-Rubio, Real time detection of malicious DoH traffic using statistical analysis, Comput. Netw. 234 (2023) 109910.

[18] M. Moure-Garrido, S.K. Das, C. Campo, C. Garcia-Rubio, Real-time analysis of encrypted DNS traffic for threat detection, in: ICC 2024-IEEE International Conference on Communications, IEEE, 2024, pp. 3292–3297.

[19] M. Zuo, C. Guo, H. Xu, Z. Zhang, Y. Cheng, METC: A hybrid deep learning framework for cross-network encrypted dns over HTTPS traffic detection and tunnel identification, Inf. Fusion 121 (2025) 103125.

[20] M. Long, Y. Cao, J. Wang, M. Jordan, Learning transferable features with deep adaptation networks, in: International Conference on Machine Learning, PMLR, 2015, pp. 97–105.

[21] Y. Zhu, F. Zhuang, J. Wang, G. Ke, J. Chen, J. Bian, H. Xiong, Q. He, Deep subdomain adaptation network for image classification, IEEE Trans. Neural Networks Learn. Syst. 32 (4) (2020) 1713–1722.

[22] Y. Ganin, V. Lempitsky, Unsupervised domain adaptation by backpropagation, in: International Conference on Machine Learning, PMLR, 2015, pp. 1180–1189.

[23] M. Long, Z. Cao, J. Wang, M.I. Jordan, Conditional adversarial domain adaptation, Adv. Neural Inf. Process. Syst. 31 (2018) 1640–1650.

[24] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, T. Darrell, Tent: Fully test-time adaptation by entropy minimization, in: International Conference on Learning Representations, ICLR, 2021, pp. 1–15.

[25] Y. Iwasawa, Y. Matsuo, Test-time classifier adjustment module for model-agnostic domain generalization, Adv. Neural Inf. Process. Syst. 34 (2021) 2427–2440.

[26] S. Siby, M. Juarez, C. Diaz, N. Vallina-Rodriguez, C. Troncoso, Encrypted dns→privacy? A traffic analysis perspective, in: Proceedings of the Network and Distributed System Security Symposium, NDSS, 2020, pp. 1–18.

[27] R. Xie, Y. Wang, J. Cao, E. Dong, M. Xu, K. Sun, Q. Li, L. Shen, M. Zhang, Rosetta: Enabling robust tls encrypted traffic classification in diverse network environments with tcp-aware traffic augmentation, in: Proceedings of the ACM Turing Award Celebration Conference-China 2023, 2023, pp. 131–132.

[28] J. Liang, R. He, T. Tan, A comprehensive survey on test-time adaptation under distribution shifts, Int. J. Comput. Vis. 133 (1) (2025) 31–64.

[29] Y. Feng, J. Jiang, M. Tang, R. Jin, Y. Gao, Rethinking supervised pre-training for better downstream transferring, in: International Conference on Learning Representations, ICLR, 2022, pp. 1–22.

[30] Majestic, The Majestic Million, 2026, https://majestic.com/reports/majestic-million. (Accessed: 21 May 2026).

[31] X. Hu, C. Zhu, G. Cheng, R. Li, H. Wu, J. Gong, A deep subdomain adaptation network with attention mechanism for malware variant traffic identification at an IoT edge gateway, IEEE Internet Things J. 10 (5) (2022) 3814–3826.

[32] J. Bang, H. Kim, Y. Yoo, J.-W. Ha, J. Choi, Rainbow memory: Continual learning with a memory of diverse samples, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 8218–8227.

[33] T. Gong, Y. Kim, T. Lee, S. Chottananurak, S.J. Lee, Sotta: Robust test-time adaptation on noisy data streams, Adv. Neural Inf. Process. Syst. 36 (2023) 14070–14093.

[34] X. Li, B. Tang, H. Li, Adaer: An adaptive experience replay approach for continual lifelong learning, Neurocomputing 572 (2024) 127204.

[35] Y. Yu, L. Sheng, R. He, J. Liang, STAMP: Outlier-aware test-time adaptation with stable memory replay, in: European Conference on Computer Vision, Springer, 2024, pp. 389–407.

[36] J. Ma, Improved self-training for test-time adaptation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 23701–23710.

[37] H. Lu, J. Liu, J. Peng, J. Lu, Adversarial attacks based on time-series features for traffic detection, Comput. Secur. 148 (2025) 104175.

[38] A. Niakanlahiji, S. Orlowski, A. Vahid, J.H. Jafarian, Toward practical defense against traffic analysis attacks on encrypted DNS traffic, Comput. Secur. 124 (2023) 103001.

[39] M. Wang, N. Yang, N.J. Forcade-Perkins, N. Weng, Progen: Projection-based adversarial attack generation against network intrusion detection, IEEE Trans. Inf. Forensics Secur. 19 (2024) 5476–5491.

[40] S. Li, Y. Cao, S. Liu, Y. Lai, Y. Zhu, N. Ahmad, Hda-ids: A hybrid dos attacks intrusion detection system for iot by using semi-supervised cl-gan, Expert Syst. Appl. 238 (2024) 122198.

[41] F.A. Croitoru, V. Hondru, R.T. Ionescu, M. Shah, Diffusion models in vision: A survey, IEEE Trans. Pattern Anal. Mach. Intell. 45 (9) (2023) 10850–10869.

[42] L. Yang, Z. Zhang, Y. Song, S. Hong, R. Xu, Y. Zhao, W. Zhang, B. Cui, M.-H. Yang, Diffusion models: A comprehensive survey of methods and applications, ACM Comput. Surv. 56 (4) (2023) 1–39.



Yandu Yi received the B.S. degree in Cyber Science and Engineering from Southeast University, Nanjing, China, in 2024. She is currently a Master Candidate with the School of Cyber Science and Engineering, Southeast University. Her research interests include DoH traffic identification and network security.



Xiaoyan Hu received her Ph.D. in computer architecture from Southeast University, Nanjing, China, in 2015. She visited the NetSec Laboratory, Colorado State University, from 2010 to 2012. She is currently an Associate Professor with the School of Cyber Science and Engineering, Southeast University. Her research interests include website fingerprinting, network intrusion detection, network security, and future network architecture.



Miao Yang received the B.S. degree in Electronic Commerce from the Shandong University of Finance and Economics, Jinan, China, in 2019, and the M.S. degree in Cyber Science and Engineering from Southeast University, Nanjing, China, in 2025. His research interests include DoH traffic identification and network security.



Guang Cheng received the Ph.D. degree in computer science from Southeast University, Nanjing, China, in 2003. From 2006 to 2007, he held a post-doctoral position with the School of Electrical and Computer Engineering, Georgia Institute of Technology. He is currently a Full Professor with the School of Cyber Science and Engineering, Southeast University. His research interests include network security, network measurement, and encrypted traffic analysis.



Ruidong Li received the B.S degree in engineering from Zhejiang University, China, in 2001, and the M.S. and Ph. D. degrees from the University of Tsukuba, in 2005 and 2008, respectively. He was a Researcher with the Network Architecture Laboratory, National Institute of Information and Communications Technology (NICT). He is currently an Associate Professor with Institute of Science and Engineering, Kanazawa University. His current research interests include Internet of Things, future networks, information-centric network, security/secure architectures of future networks, and next-generation wireless network. He serves on the Editorial Board of IEEE Internet of Things Journal and KSII Transactions on Internet and Information Systems.



Hua Wu received the Ph.D. degree in computer application technology from Southeast University, Nanjing, China, in 2010. She is currently an Associate Professor with the School of Cyber Science and Engineering, Southeast University. Her research interests include computer network security and encrypted traffic analysis.